

Laravel Developer Assignment

Lead Management Module — Laravel 12

Candidate:	Gopi Patoliya
Role:	PHP Developer (Laravel)
Estimated Time:	3 – 4 hours
Submission:	GitHub repository link

Overview

Build a Lead Management module in a fresh Laravel 12 application. The task covers project setup, authentication, full CRUD operations, search/filtering, quick status updates, lead assignment, and CSV export. This assignment is designed to verify hands-on Laravel fundamentals — Artisan usage, resource controllers, Form Request validation, and clean query building. Given your background delivering multi-tenant HRMS, CRM, and property/booking platforms in Laravel, this task should map closely onto patterns you've already built in production (role-based access, RESTful resource structures, and reporting/export flows).

1. Fresh Laravel 12 Installation

- Install a fresh Laravel 12 project using Composer: `composer create-project laravel/laravel lead-management`
- Configure the `.env` file (database credentials, app URL, etc.)
- Run `php artisan migrate` and confirm the default welcome page loads correctly

2. Authentication — Login / Logout

- Implement basic authentication (Laravel Breeze, or manual implementation using the Auth facade — candidate's choice; mention which was used and why in the README)
- Only logged-in users should be able to access any Lead module route
- Logout must properly invalidate the session

3. Lead Module — Full CRUD

Use Artisan commands to generate all core files — this is explicitly being evaluated; do not hand-create files that Artisan can generate.

```
php artisan make:model Lead -mcr
php artisan make:request StoreLeadRequest
php artisan make:request UpdateLeadRequest
```

Migration — leads table

- id
- name
- email
- phone
- company_name

- status — enum: new, contacted, converted, lost
- source — enum: website, referral, social_media, cold_call, other
- assigned_to — nullable, foreign key to users.id (staff member who owns the lead)
- notes — nullable, text
- timestamps

CRUD routes to implement

Action	Route	Description
List (page)	GET /leads	Blade page shell (table container, filters, no server-rendered rows)
List (data)	GET /leads/data	AJAX endpoint — returns JSON (paginated) used to populate the table
Add	GET /leads/create, POST /leads	Form to add a new lead
Edit	GET /leads/{lead}/edit, PUT /leads/{lead}	Form to update a lead
Delete	DELETE /leads/{lead}	Delete a lead via AJAX (with confirmation, no page reload)
View	GET /leads/{lead}	Show single lead details

Use a resource controller via `Route::resource('leads', LeadController::class)` for the standard CRUD actions, plus the two extra custom routes above (`GET /leads/data` and `PATCH /leads/{lead}/status`) for the AJAX-driven pieces.

AJAX requirements

- Index page (`GET /leads`) should render only the page shell — filters, search box, and an empty table/container
- On page load (and whenever search/filter/pagination changes), call `GET /leads/data` via `fetch` or `Axios` and render rows from the JSON response — no full page reload for browsing
- `GET /leads/data` should return JSON with the paginated leads plus pagination meta (`current_page`, `last_page`, `total`, etc.), applying the same search/filter query as before
- Delete (`DELETE /leads/{lead}`) must be triggered via AJAX: show a confirmation dialog, call the endpoint, then remove the row from the DOM (or re-fetch) on success — the page must not reload
- Show a loading indicator while the AJAX list request is in flight, and toast/flash-style feedback for delete success or failure
- Handle AJAX error responses gracefully (e.g. 404 if already deleted, 403 if unauthorized per the Policy) and surface a message to the user instead of failing silently

4. Search & Filter

- Add a search box on the index page — search by name, email, or phone
- Add filter dropdowns for status and source
- Preserve search/filter values across pagination links (e.g. `?search=...&status=...`)

5. Status Update (Quick Action)

- Allow changing a lead's status directly from the index page (dropdown or button), without opening the full edit form
- Implement via a separate route: `PATCH /leads/{lead}/status`

6. Lead Assignment

- Add a way to assign a lead to a user (assigned_to) — dropdown of users on the create/edit form
- Show the assigned user's name on the index and show views

7. Export to CSV

- Add an "Export to CSV" button on the index page
- Export should respect the currently applied search/filter values (not the entire table every time)

8. Validation & UX

- Use Form Request classes for all validation — not `$request->validate()` inline in the controller
- Use flash messages for success/error feedback (e.g. "Lead created successfully")
- Show validation errors properly in Blade forms, with old input retained on failure

9. Authorization (Basic)

- Only the user who created a lead (or an admin) should be able to delete it
- Implement this via a simple Policy or Gate — not an inline role check scattered across views/controllers

10. Custom Helper Function

Create one custom, reusable helper function for this module — do not rely on inline logic in Blade or the controller for it.

- Create `app/Helpers/LeadHelper.php` and register it (autoload via `composer.json` "files" array, or a small `HelperServiceProvider` — candidate's choice; document which approach and why in the README)
- Implement `leadStatusBadge($status)`, returning the Bootstrap/Tailwind class + label for new / contacted / converted / lost
- Use this helper consistently everywhere a status is displayed — index table and show view — instead of repeating if/switch logic in the Blade files
- It should be a plain function (not a static class method called awkwardly from Blade) so it can be called directly in views, e.g. `{{ leadStatusBadge($lead->status) }}`
- Run `composer dump-autoload` after registering the helpers file and confirm it loads correctly

Evaluation Criteria

Area	What to look for
Artisan usage	make:model -mcr, make:request, make:policy actually used, not hand-written
Controller	Resource controller used correctly; no leftover unused methods
Validation	Proper Form Request classes, not inline validation
Auth	auth middleware correctly applied to all lead routes
Search / filter	Clean query building (scopes or chained query builder), no if-else soup
Export	CSV generated properly (not raw HTML dumped as .csv)
Authorization	Policy used correctly, not role checks scattered in views
Migration	Clean column types, correct foreign keys and defaults
Routes	RESTful — Route::resource plus custom routes for data/status/export
AJAX (list)	GET /leads/data returns clean JSON; index page has no server-rendered rows; search/filter/paging
AJAX (delete)	Delete confirmed and executed via AJAX; row removed from DOM/re-fetched on success; errors
Helper function	leadStatusBadge() helper in app/Helpers, properly autoloaded, used consistently in views instead

Deliverable

- GitHub repository link
- README.md with setup steps (.env configuration, php artisan migrate, default login credentials if seeded)
- README.md should also note: which auth approach was used, how the Helper file is autoloaded (composer.json files array vs. service provider), and confirm composer dump-autoload was run
- A seeder with a few dummy leads and users for testing (optional but recommended)

Note: Please complete this task independently and submit a working GitHub repository link with clear commit history. Assumptions made during implementation should be documented in the README.